Name: Ritesh Ravindra Solanke

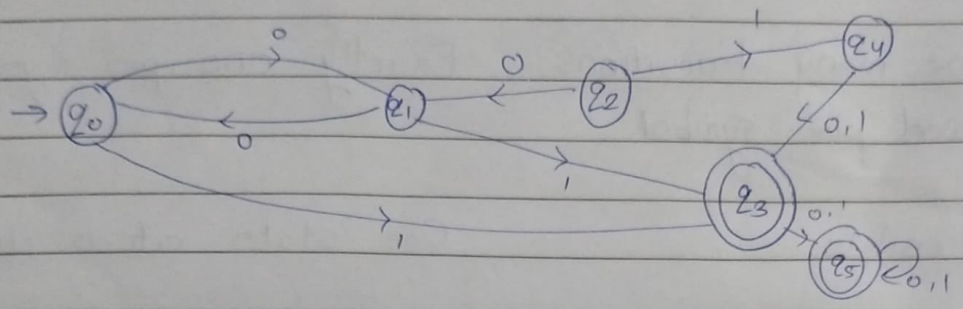
Roll no: IT2262

RBT24IT126

FLAT

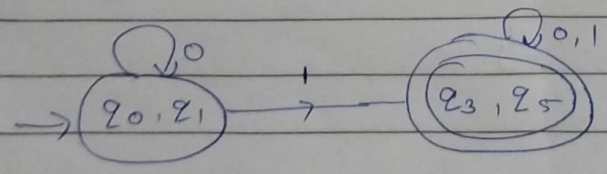
Assignment - 1

Q1. Construct a minimum state.



State \ Symbol	0	1
q_0	q_1	$*q_3$
q_1	q_0	$*q_3$
q_2	— dead state —	
$*q_3$	$*q_5$	$*q_5$
q_4	— dead state —	
$*q_5$	$*q_5$	$*q_5$

$P_0 = \{q_0, q_1\}, \{q_3, q_5\}$



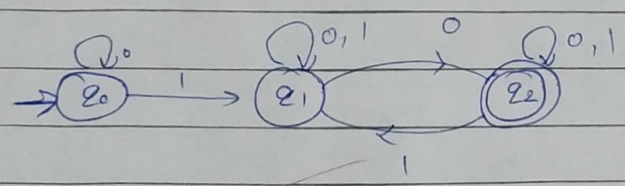
State \ Symbol	0	1
q_0, q_1	q_0, q_1	$*q_3, q_5$
$*q_3, q_5$	$*q_3, q_5$	$*q_3, q_5$

Q. Differentiate NFA & DFA.

⇒

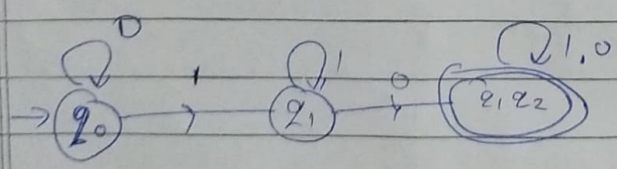
NFA	DFA
Zero, one or many transitions are allowed per symbol.	Exactly one per symbol
Multiple possible.	One state at a time
~~Easy~~ Hard implementation	Easy implementation
ε moves are allowed	ε moves not allowed.

Q3. Convert NFA to DFA.



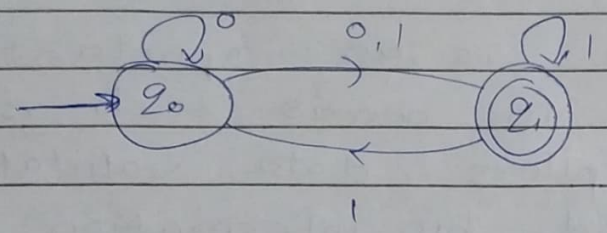
⇒

	0	1
q_0	q_0	q_1
q_1	$\{q_1, q_2\}$	q_1
q_2	q_2	$\{q_1, q_2\}$



Page No.
Date

Q4. Convert NFA to DFA.

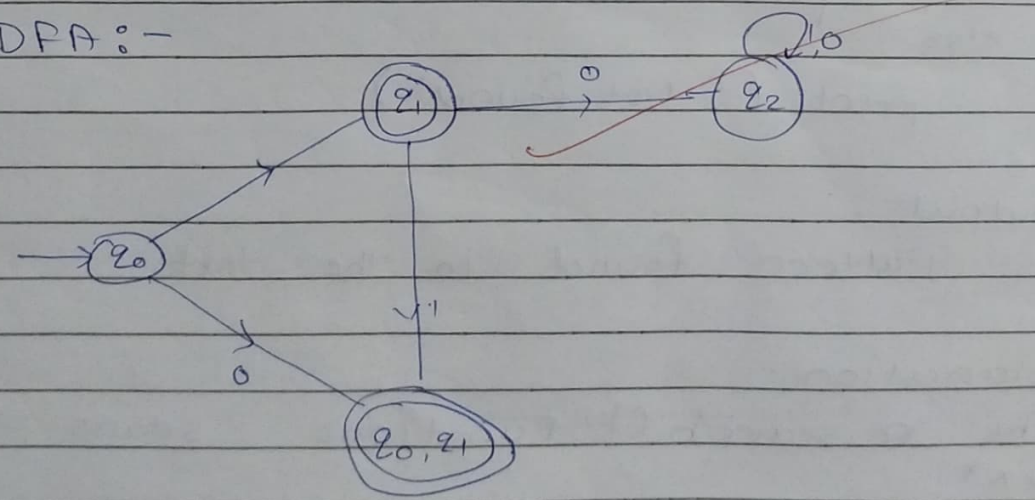


	0	1
q_0	$\{q_0, q_1\}$	q_1
q_1	ϕ	$\{q_0, q_1\}$

Transition Table (T') for DFA

	0	1
q_0	$\{q_0, q_1\}$	q_1
* q_1	ϕ	$\{q_0 q_1\}$
*(q_0, q_1)	$\{q_0, q_1\}$	$\{q_0\ q_1\}$

DFA :-



Page No.
Date

Q6. Case Study:

⇒ Pattern recoginition is an important expression It is widely used in areas such as text, searching, compilers, data validation, spam detection and bio information. In this case study, we demonstrate how patterns can be recognized in text using a programming language.

Code: python

```
import re

text = " Finite automata is an important topic
         in computer science".
pattern = " automata".
if research(pattern.txt):
        print(" Pattern ~~not~~ found in the text")
else:
    print (" Not found")
```

Output:
Pattern found in the text.

- Explanation:

i) The re.search() Functions scans the entire text.

ii) If the given pattern appears anywhere in the text, it returns a match.

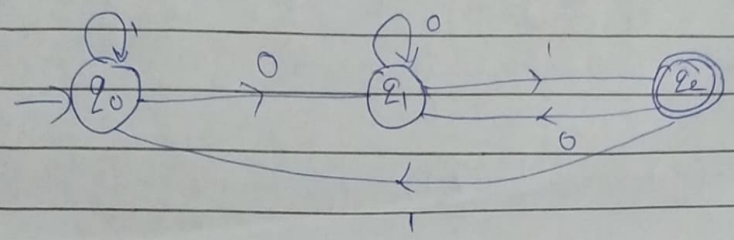
iii) This behaviour is similar to a DFA which moves through states and reaches •

an accepting state where the pattern is detected.

iv. If no match is found, the input string is rejected.

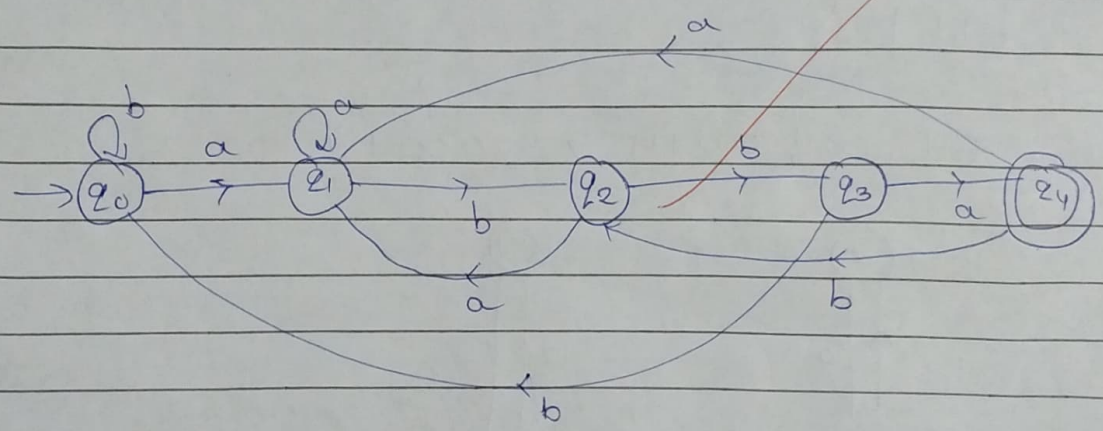
Q7. DFA with "01" $\Sigma = \{0, 1\}$

⇒

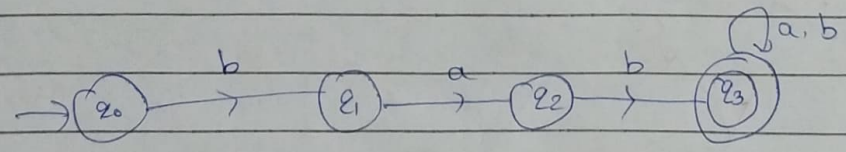


Q8 DFA 'abba'

⇒



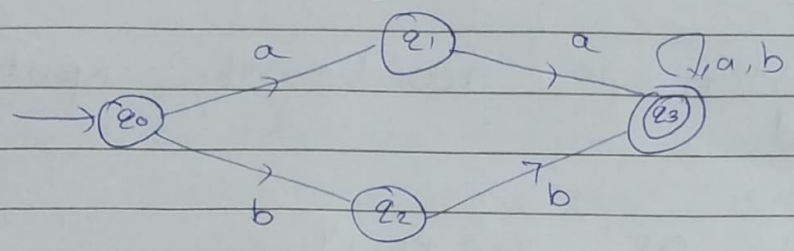
Q9. NFA starting with bab.



Page No.
Date

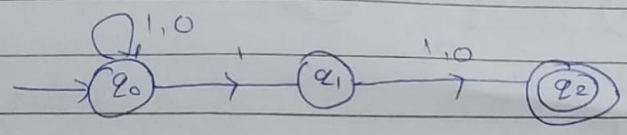
Q10 NFA starting with "aa" or "bb"

=>



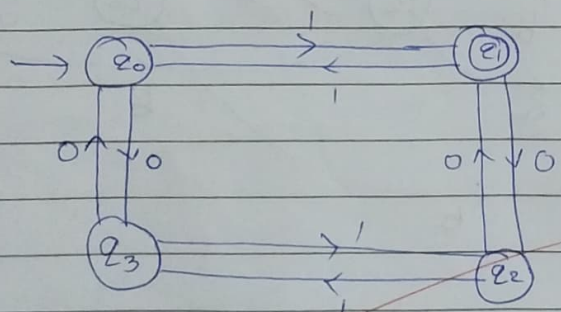
Q11 NFA where second last bit is 1.

=>



Q12. DFA for strings with even 0's and odd '1's'

=> L = { 1, 001, 00111 > 00001 }



Bupti